

# Athena

Autonomous Product Intelligence & Decision Support System

Architecture Documentation

Ingestion · Hybrid retrieval (dense + BM25 + RRF + rerank)  
Multi-agent deep research · Long-term memory · Knowledge graph  
Evaluation & observability · Google Gemini

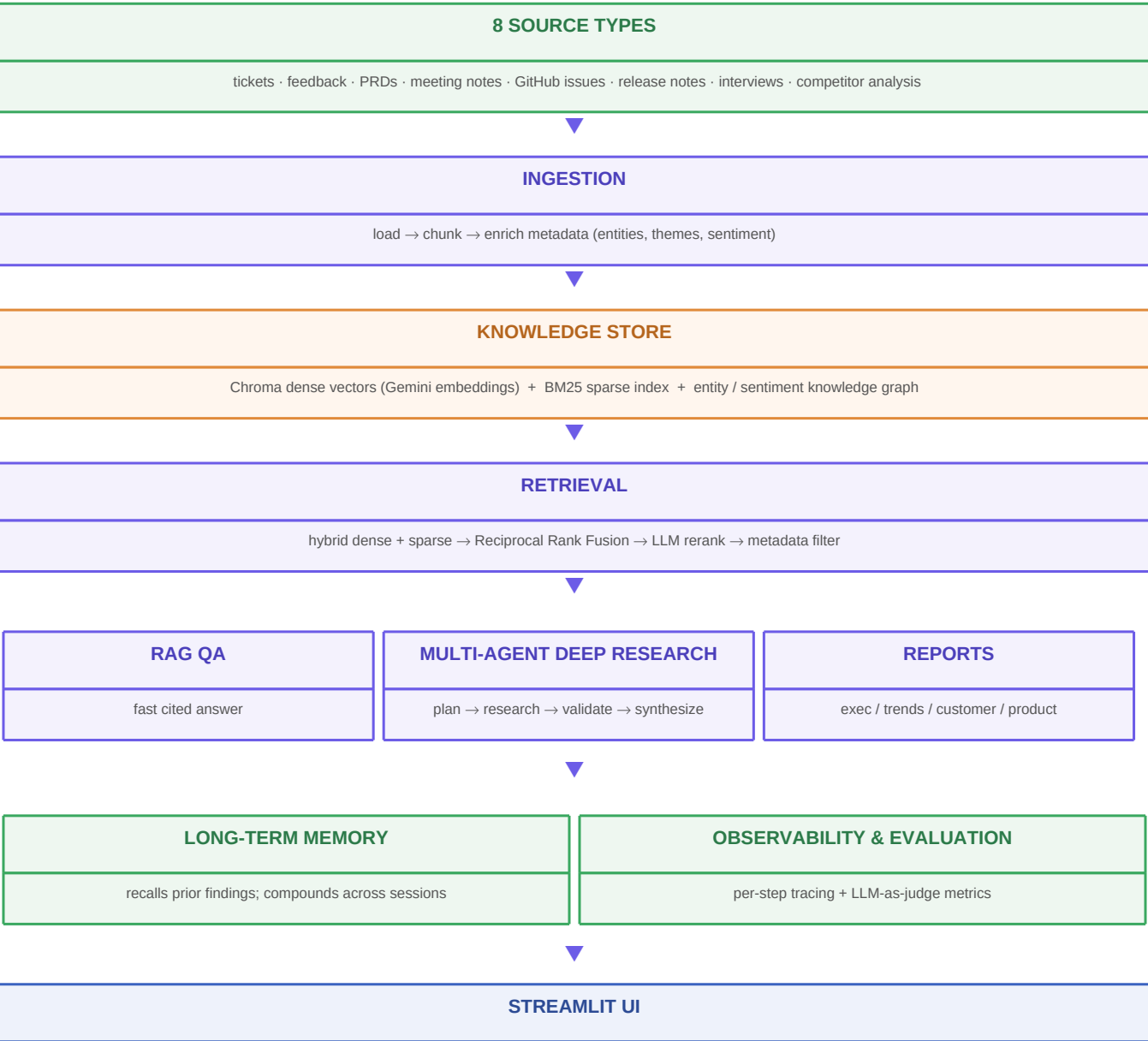
## 1. Design goals

The task asks for a system that behaves like an **autonomous knowledge worker**, not a chatbot. Concretely it must: understand, retrieve, reason, plan, research, validate, remember, and report — with explainable, evidence-backed answers.

That shaped five guiding principles:

1. **Evidence-first.** Nothing is asserted without a retrievable source. Every factual claim is cited and resolvable back to its exact chunk.
2. **Transparent agency.** Agent control flow is explicit code with per-step tracing — not a black box. Reliability and explainability beat "magic".
3. **Hybrid retrieval.** Business questions mix semantic and lexical intent, so retrieval combines dense + sparse and reranks.
4. **Compounding knowledge.** Long-term memory and a knowledge graph let insight accumulate across sessions and connect across sources.
5. **Reproducible & runnable.** Local-first stores, a deterministic dataset, and one API key — anyone can run the whole thing.

## 2. System overview



### 3. Component design

#### 3.1 Ingestion ([athena/ingestion/](#))

| Stage    | File                                | What it does                                                                                                                                                                                  |
|----------|-------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Generate | <a href="#">generate_dataset.py</a> | Deterministic synthetic corpus for a fictional SaaS ("Flowdesk"): 281 docs across 8 source types, spanning ~12 months, with <b>shared entities</b> so cross-source reasoning has real signal. |
| Load     | <a href="#">loaders.py</a>          | Reads JSON (structured) or <a href="#">.md/.txt</a> (user drop-ins) into typed <a href="#">Documents</a> .                                                                                    |
| Chunk    | <a href="#">chunker.py</a>          | Paragraph-aware, size-bounded chunking with overlap. Respects document structure (headings/sections stay together).                                                                           |
| Enrich   | <a href="#">enrich.py</a>           | Attaches entities, themes, sentiment. <b>Derives</b> from structured metadata when present (free, deterministic); falls back to an <b>LLM</b> extractor for unstructured inputs.              |

#### 3.2 Knowledge store & retrieval ([athena/retrieval/](#))

- [vector\\_store.py](#) — Chroma persistent client. We embed with Gemini ([gemini-embedding-001](#)) using **asymmetric task types** ([RETRIEVAL\\_DOCUMENT](#) for indexing, [RETRIEVAL\\_QUERY](#) for search), which improves recall vs. symmetric embeddings. Chroma stores vectors + metadata and does cosine ANN.
- [sparse\\_index.py](#) — BM25 (rank-bm25). Captures exact tokens dense retrieval misses: "SSO", "SCIM", "SAML", error codes, customer/version strings.
- [knowledge\\_graph.py](#) — co-occurrence graph of entities ■ documents, entity ■ entity, and entity ■ sentiment. Powers "connect the dots" queries and aggregate report statistics.
- [hybrid.py](#) — the retrieval pipeline:
  - dense search + sparse search (candidates)
  - **Reciprocal Rank Fusion** (rank-based, so incomparable score scales don't matter)
  - **LLM cross-encoder rerank** (Gemini flash) to pick the top-N truly relevant
  - metadata filtering by [source\\_type](#) (from agent hints)
- [knowledge\\_base.py](#) — a single facade ([build\(\)](#) / [load\(\)](#) / [retrieve\(\)](#)) used by everything downstream.

#### 3.3 RAG QA ([athena/agents/rag.py](#))

Single-shot path for simple questions: retrieve → build cited evidence set → grounded generation. Resolves [\[E#\]](#) citations in the answer back to source labels. Used by the **Chat** tab and the **evaluation** harness.

#### 3.4 Multi-agent deep research ([athena/agents/](#))

An explicit state machine:

RECALL MEMORY → PLAN → RESEARCH (per sub-question) → VALIDATE → SYNTHESIZE → REMEMBER

| Agent | File | Role |
|-------|------|------|
|-------|------|------|

|              |                                 |                                                                                                                                                   |
|--------------|---------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| Planner      | <a href="#">planner.py</a>      | Decomposes the question into ≤6 researchable sub-questions with source hints; primed with recalled memory.                                        |
| Researcher   | <a href="#">researcher.py</a>   | Retrieves evidence per sub-question and produces specific, cited findings. Runs independently per sub-question.                                   |
| Validator    | <a href="#">validator.py</a>    | Independently checks each finding against its evidence; drops unsupported claims, flags partial ones. <b>This is the anti-hallucination gate.</b> |
| Synthesizer  | <a href="#">synthesizer.py</a>  | Weaves validated findings into a coherent, cited answer with recommended actions (uses Gemini <b>Pro</b> for long-context reasoning).             |
| Orchestrator | <a href="#">orchestrator.py</a> | Drives the state machine, merges evidence, emits streaming progress events, and traces every step.                                                |

**Citation integrity:** Researchers cite evidence with local `E#` ids, which are immediately resolved to **durable `chunk_ids`**. When evidence from all sub-questions is merged, findings remain valid and the synthesizer/validator see a consistent `E#` space re-derived from the merged set. No citation is ever fabricated or silently dropped.

### 3.5 Long-term memory ([athena/memory/long\\_term.py](#))

JSONL-backed store + in-memory embedding index. Stores distilled findings from research runs and user preferences. `recall()` does semantic top-k retrieval and feeds the planner, so discoveries compound across sessions.

### 3.6 Reports ([athena/reports/generator.py](#))

Four report types (executive, trends, customer, product). Each combines knowledge-graph **aggregate stats** + retrieved **evidence** and asks Gemini Pro for a polished, cited, leadership-ready Markdown document.

### 3.7 Evaluation & observability ([athena/eval/](#), [athena/core/tracing.py](#))

- **Tracing:** every operation is a span (llm/retrieval/agent) with timing, inputs/outputs, and token estimates, persisted as JSON and shown in the UI's **Traces** tab. Zero external dependencies.
- **Eval harness:** a golden set of business questions scored on retrieval source coverage, keyword recall, citation coverage, and **LLM-as-judge** faithfulness & answer relevance. Aggregated into a report shown in the **Evaluation** tab.

### 3.8 UI ([athena/ui/app.py](#))

Streamlit app with six tabs: Chat, Deep Research (live agent progress), Reports, Knowledge Explorer (graph), Evaluation, and Traces.

## 4. Model routing

| Task                                     | Model                                 | Why                                                    |
|------------------------------------------|---------------------------------------|--------------------------------------------------------|
| Agents, retrieval reranking, RAG answers | <a href="#">gemini-2.5-flash</a>      | Fast, cheap, strong for agentic/tool reasoning.        |
| Synthesis & report writing               | <a href="#">gemini-2.5-pro</a>        | Best long-context reasoning for connecting insights.   |
| Bulk metadata enrichment                 | <a href="#">gemini-2.5-flash-lite</a> | Cheapest for high-volume extraction.                   |
| Embeddings                               | <a href="#">gemini-embedding-001</a>  | One provider for LLM + vectors; asymmetric task types. |

All IDs are centralized in [athena/core/config.py](#) and overridable via env vars.

## 5. Data flow for a deep-research question

1. User asks a complex question in the **Deep Research** tab.
2. Orchestrator **recalls** related memories → primes the Planner.
3. Planner returns sub-questions (+ source hints).
4. Each Researcher **hybrid-retrieves** evidence and emits cited findings.
5. Evidence is merged & de-duplicated across sub-questions.
6. Validator judges each finding against evidence; unsupported ones are dropped.
7. Synthesizer composes the final cited answer + recommended actions.
8. Top findings are **written to memory**; the full run is **traced** to disk.
9. UI streams every step live and shows the answer, findings, and sources.

## 6. Extensibility / production swaps

The interfaces are intentionally thin so demo-grade components swap for production ones without touching business logic:

| Demo                       | Production option                |
|----------------------------|----------------------------------|
| Chroma (in-process)        | pgvector / Qdrant / Pinecone     |
| BM25 pickle                | Elasticsearch / OpenSearch       |
| JSONL memory               | Postgres / a vector DB namespace |
| Co-occurrence graph (dict) | Neo4j                            |
| File traces                | OpenTelemetry / Langfuse         |
| Streamlit                  | FastAPI + React                  |

See [TECHNICAL\\_REPORT.md](#) for trade-offs and future work.